PES UNIVERSITY

Department of Computer Science and Engineering

LAB 7: Fuzzing for Input Validation Bugs

Student Name:	A R AKSHAY KUMAR	**SRN:**	PES1UG24CS705
Course:	SE / SETA (Sem 5)	**Assigned Repo:**	SETAPESU26/L6_L7_41_laundry_service
Domain:	41 — Laundry Service	**Target Repo:**	Akshayarcot/openapi-mock-server-generator

1. Objective & Methodology

The objective of this lab is to explore **fuzz testing** as a dynamic security and quality assurance technique. By applying property-based fuzz testing with the **Hypothesis** Python library, we automatically generate random, unexpected, and malformed inputs to uncover hidden bugs (exceptions, crashes, and unhandled edge cases). Finally, we harden the script using defensive programming principles and verify complete resilience against all fuzz cases.

2. Step 1: Installing Dependencies

Hypothesis and pytest were installed in the test environment using pip:
$ pip install hypothesis pytest

3. Step 2: Running the Buggy Application

The application script (processor.py / laundry_service.py) was executed with normal happy-path inputs (regular:6, 3,5,8, 300|40). While it produces expected outputs on strictly formatted inputs, it lacks defensive validation.



4. Step 3: Fuzz Testing Implementation (test_processor.py)

We implemented fuzz test functions using Hypothesis's @given(st.text() | st.none()) strategy. This feeds hundreds of randomized Unicode strings, empty strings, delimiters, non-numeric values, and None into each target function to verify the property that **no unhandled exceptions or crashes occur**.

```
from hypothesis import given, strategies as st
from processor import parse_order, weight_average, payable_charge

@given(st.text() | st.none())
def test_parse_order_no_crash(s):
    try: parse_order(s)
    except Exception as e: assert False, f'parse_order crashed with: {e}'

@given(st.text() | st.none())
def test_weight_average_no_crash(s):
    try: weight_average(s)
    except Exception as e: assert False, f'weight_average crashed with: {e}'

@given(st.text() | st.none())
def test_payable_charge_no_crash(s):
    try: payable_charge(s)
    except Exception as e: assert False, f'payable_charge crashed with: {e}'
```

5. Step 4: Running Fuzz Tests Against Buggy Script (Failures Discovered)

Running pytest test_processor.py immediately broke all three functions. Hypothesis minimized the failure cases to expose the root input validation bugs:

Function	Failing Inputs Found	Unhandled Exception
parse_order(text)	None, '', ':'	AttributeError, ValueError: not enough values, ValueError: could not convert string to float
weight_average(csv)	None, '', '1,,3', 'a,b'	AttributeError, ValueError: invalid literal for int(), ZeroDivisionError
payable_charge(text)	None, '', '300', 'a|b'	AttributeError, ValueError: not enough values, ValueError: could not convert string to float



6. Step 5: Hardening the Application Script (Defensive Programming)

To make the functions robust against all malformed and unexpected inputs, the following defensive checks were incorporated:
• **Type Checking:** Ensure input is strictly isinstance(..., str), returning None on non-string inputs (e.g. NoneType).
• **Delimiter & Structure Verification:** Verify presence and count of delimiters (':' and '|').
• **Numeric & Domain Constraints:** Wrap numeric conversions in try...except (ValueError, TypeError) and validate that weights and charges are non-negative.
• **Division by Zero Protection:** Guard against empty lists in weight_average.

```
akshaykumar@macbook: ~/Lab 7 (Step 5: Hardened Application Execution)

$ python processor.py
==== Running parse_order ====
Enter garment:weight in kg (e.g. regular:6): regular:6
Garment and Weight: ('Regular', 6.0)

==== Running weight_average ====
Enter garment weights as CSV (e.g. 3,5,8): 3,5,8
Average Weight: 5

==== Running payable_charge ====
Enter charge|discount (e.g. 300|40): 300|40
Payable Charge: 314.0

[Info] Hardened against all edge cases: None, malformed strings, non-numerics, empty CSVs.
```

7. Step 6: Re-Running Fuzz Tests on Fixed Script (100% Passes)

We re-ran pytest test_processor.py against the hardened application. Hypothesis generated hundreds of randomized inputs across the entire Unicode space; all 3 tests passed without any crashes.

```
akshaykumar@macbook: ~/Lab 7 (Step 6: Hardened Tests Passing)

$ pytest test_processor.py
============================= test session starts ==============================
platform darwin -- Python 3.13.7, pytest-9.2.0, pluggy-1.7.0
rootdir: /Users/akshaykumar/.gemini/antigravity/scratch/L6_L7_41_laundry_service/Lab 7
plugins: hypothesis-6.170.14
collected 3 items

test_processor.py ...                                                    [100%]

============================== 3 passed in 0.28s ===============================
```

8. Deliverables & Conclusion

Deliverable	Artifact Name / Description	Status
1. PDF Document with Screenshots	PES1UG24CS705_Lab7.pdf (All 4 screenshots embedded)	**Complete**
2. Fixed Application Script	processor.py & laundry_service.py (Defensively hardened)	**Complete**
3. Updated Test Processor Code	test_processor.py & test_laundry_service.py (Hypothesis suite)	**Complete**
4. Target Repository Push	Pushed to Akshayarcot/openapi-mock-server-generator/lab 7	**Complete**

Conclusion: Fuzz testing with Hypothesis successfully identified boundary conditions, missing delimiter unpack errors, and type errors that traditional unit testing frequently misses. Defensive programming safeguards were implemented to guarantee fault tolerance, stability, and security in production software.